

SeraEdit: Reliable Language-Guided MusicXML Editing through Structured Score Patches

[AUTHOR GIVEN NAME] [AUTHOR FAMILY NAME]^{a,*}

^a[AFFILIATION, POSTAL ADDRESS, COUNTRY]

Abstract

SeraEdit is local-first research software for reliable language-guided editing of symbolic scores. Rather than asking a language model to rewrite an entire MusicXML document, it represents a request as a versioned ScorePatch bound to stable event identifiers, target and protected scopes, and a source fingerprint. Layered validators check schema, score structure, duration, notation relations, protected content, and MusicXML round-trip fidelity inside an atomic transaction. A desktop interface and MuseScore bridge expose proposals, diffs, rejection reasons, and undo without silently overwriting the host score. The release includes synthetic scores, editing tasks, three evaluation conditions, resumable experiment tooling, and offline verification fixtures.

Keywords: MusicXML, symbolic music, score editing, structured patches, validation, research software

1. Motivation and significance

MusicXML is a widely used interchange representation for common Western music notation, allowing scores to move among notation, analysis, engraving, and archival systems [1]. Natural-language interfaces could reduce the mechanical cost of local edits such as transposition, dynamics, articulation, or voice assignment. However, asking a model to return a complete rewritten MusicXML document mixes the intended change with thousands of unrelated elements. A syntactically valid response can still alter an unselected staff, break a tie, change measure duration, or drift from the source the user reviewed.

*Corresponding author

Email address: [SUPPORT EMAIL] ([AUTHOR GIVEN NAME] [AUTHOR FAMILY NAME])

Nr.	Code metadata description	Metadata
C1	Current code version	1.0.0-dev.14
C2	Permanent repository link	https://github.com/Selancee/Sera ; must be public and tagged before submission
C3	Reproducible capsule	[ZENODO OR CODE OCEAN DOI/URL]
C4	Legal code license	MIT; benchmark data CC0-1.0
C5	Versioning system	Git
C6	Languages, tools and services	Python, TypeScript/JavaScript, React, FastAPI, Electron, QML, PowerShell, MusicXML
C7	Build, environment and dependencies	Python ≥ 3.10 ; Node.js/npm for interface development; Windows 10/11 desktop; requirements, tested Windows constraints and npm lock files
C8	Developer documentation/manual	docs/softwarex/ , reviewer guide and generated OpenAPI at /docs
C9	Support email	[SUPPORT EMAIL]

Table 1: Code metadata (mandatory).

Nr.	Executable software metadata description	Metadata
S1	Current version	1.0.0-dev.14
S2	Permanent executable link	[RELEASE ASSET URL]
S3	Reproducible capsule	[ZENODO OR CODE OCEAN DOI/URL]
S4	Legal software license	MIT
S5	Platforms	Windows 10/11 x64; source-level core uses Python ≥ 3.10
S6	Installation requirements	docs/softwarex/INSTALLATION.md
S7	User manual	docs/softwarex/USER_MANUAL.md
S8	Support email	[SUPPORT EMAIL]

Table 2: Executable software metadata (optional).

¹¹ Symbolic-music toolkits provide programmatic analysis and generation [2, 3],
¹² while recent work studies music reasoning and generation with language
¹³ models [4, 5]. SeraEdit addresses a narrower systems problem: making a
¹⁴ language-guided edit inspectable, source-bound, minimal, and rejectable be-
¹⁵ fore it returns to a professional notation environment. The research object
¹⁶ is therefore a transaction over a canonical score representation, not an auto-

17 matically accepted generated score.

18 The software supports controlled research on constrained generation, deter-
19 ministic evaluation of editing agents, repair/refusal policies, and human-in-
20 the-loop notation workflows. Three separately implemented conditions com-
21 pare full-document rewriting, unprotected structured patches, and a fully
22 validated transaction without changing source scores or metric code. This re-
23 sembles structured code-editing evaluation [6], but adds musical-time, voice,
24 and notation-relation invariants. Citation metadata follows software-citation
25 principles [7].

26 **2. Software description**

27 *2.1. Software architecture*

28 Figure 1 shows the local pipeline. A MusicXML snapshot is imported into
29 one canonical **ScoreDocument**. Editable events receive stable identifiers, and
30 a canonical SHA-256 fingerprint identifies the exact source state. Rendering,
31 preview, patch execution, MIDI conversion, and export consume the same
32 document.

33 Applying a patch validates the source fingerprint and schema, resolves se-
34 lectors and preconditions, clones the score, applies operations, validates the
35 result, exports and re-imports MusicXML, and finally commits or rolls back.
36 The result contains normalized errors, a human-readable diff, fingerprints,
37 and snapshots for undo/redo. A transaction failure cannot leave a partially
38 modified score.

39 *2.2. Software functionalities*

40 A **ScoreScope** selects measures, parts, staves, voices, event identifiers, and
41 optional time ranges. Every patch declares editable and protected regions.
42 Operations use a versioned JSON schema and include selectors, arguments,
43 preconditions, and expected change counts. The research schema supports
44 pitch and duration changes, insertions and deletions, dynamics, articulations,
45 ties, slurs, signature changes, voice moves, motif duplication, chord replace-
46 ment, and transactional batches. The product enables only operation families
47 with an established host round-trip contract; unknown operations fail closed.
48 Validators cover schema, structural references, exact measure duration, nota-
49 tion relations, semantic constraints, protected content, and MusicXML round
50 trips. Deterministic repair is restricted to transparent serialization correc-
51 tions. Optional model repair is bounded and receives immutable scopes plus
52 explicit errors; it cannot bypass the transaction.

53 Provider adapters record model, latency, token usage, estimated cost, re-
54 quest identifier, finish reason, raw output, parsed output, and normalized

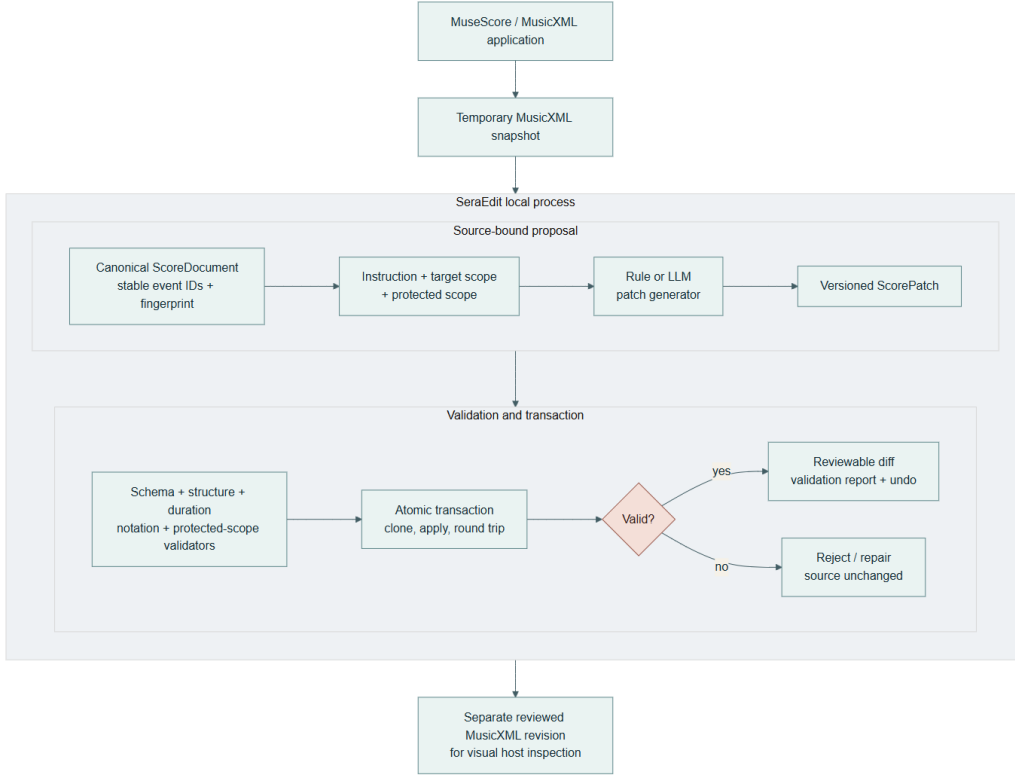


Figure 1: SeraEdit component and transaction architecture. The notation host remains the visual source. SeraEdit produces a separate reviewed revision or rejects the transaction while retaining the source.

55 errors. Secrets stay outside source and experiment artifacts. The desktop
 56 can return local candidates before an optional asynchronous model refine-
 57 ment completes. Neither path auto-applies a result.
 58 The evaluation subsystem contains Full Rewrite, Patch Only, and Sera Full
 59 conditions. Runs preserve configuration, prompt, benchmark and depen-
 60 dency hashes, request caches, raw/normalized outputs, metrics, error records,
 61 and resume state. Metrics are recomputed without another provider call.

62 3. Illustrative examples

63 For “Transpose measures 1–2 of staff 1 up two semitones and preserve rhythm;
 64 leave staff 2 unchanged,” SeraEdit binds the proposal to the imported fin-
 65 gerprint and resolves only pitched events in the requested region. Preview
 66 applies the transposition to a clone. Duration equality is checked with rati-
 67 onal values, protected staff 2 is compared event-by-event, and exported
 68 MusicXML must parse, import, export, and parse again. A valid preview

69 reports the exact changed count; stale fingerprints, missing events, duration
 70 changes, or protected differences reject and roll back the patch.
 71 The optional MuseScore Studio 4 bridge exports a temporary snapshot and
 72 selection to the local Sera process. After review, it opens a separate revised
 73 file. This is not in-place control of MuseScore’s internal document: the orig-
 74 inal open score is not overwritten and final visual inspection remains a user
 75 action.
 76 The offline core split contains 120 tasks over 20 short CC0 synthetic scores,
 77 spanning pitch, rhythm, key and harmony, voice and texture, dynamics, in-
 78 sersion and deletion, relations, meter, compound edits, and expected refusals.
 79 Each task declares target/protected scope and deterministic constraints. The
 80 fixture provider returns known outputs to exercise all runner and metric paths
 81 without network access or token cost.
 82 For review, `scripts/run_reviewer_demo.py` runs six representative tasks
 83 through the actual local product entry point and writes a compact report
 84 plus five host-openable MusicXML revisions (the sixth task is an expected
 85 refusal). It needs no API key or network connection. The included Windows
 86 CI workflow repeats the Python tests, benchmark validation, reviewer demo,
 87 package audit, frontend tests, and build.
 88 On Windows 11 build 10.0.26200 with Python 3.12.5, Node.js 24.16.0 and
 89 npm 11.13.0, automatic benchmark validation accepted 120/120 task def-
 90 initions. Human inspection completed 120/120 current primary decisions
 91 and a stratified 30/30 repeat check with zero stale records; all current deci-
 92 sions were compliant after correction. The append-only export retains 194
 93 records, including superseded findings. Both passes used the same pseudony-
 94 mous reviewer, so this verifies task/Gold semantics and host-visible no-
 95 tation but not inter-rater reliability. The Python suite passed 408 tests.
 96 Vitest passed 120 tests in 72 files; the Vite production build transformed
 97 216 modules; and staged backend, compatibility-launcher and Electron run-
 98 time smoke tests passed. A fresh `softwarex_verification_120_v1` fix-
 99 ture run completed 360/360 task-condition cases with zero execution er-
 100 rors. Reproducibility checks confirmed configuration, benchmark, prompt,
 101 dependency, evidence, metric-row and run-count integrity. These are soft-
 102 ware verification results, not model performance: the manifest explicitly sets
 103 `formal_results_allowed=false`.
 104 We also replayed every instruction through the interactive product entry
 105 point without supplying its Gold patch to generation. Across English and
 106 Chinese instructions and three independent repetitions, 720/720 runs com-
 107 pleted generation or expected refusal, transaction preview, atomic commit,
 108 protected-scope checks, deterministic constraints, and MusicXML export/re-
 109 import. All 660 executable runs produced valid MusicXML; 60 conflict runs

110 refused safely; no unsafe execution occurred. Patch and result fingerprints
 111 were identical in all 240 repeated task-language groups. After correcting
 112 a Chinese preserve-pitch/tenuto ambiguity exposed by cross-language com-
 113 parison, all 120 task groups also produced semantically equivalent patches
 114 and identical English/Chinese final score fingerprints. The compact snap-
 115 shot preserves all metrics and hashes for all 1,380 raw and host evidence files
 116 plus 220 host-openable outputs for complete task review. This is determin-
 117 istic product-path acceptance, not remote-model accuracy or human musical
 118 quality.

119 We separately tested localization when the notation host supplied a wider
 120 selection than the measure named in the instruction. The robustness run-
 121 ner added one adjacent measure to each eligible explicit-measure request. It
 122 passed 240/240 bilingual runs; expansion was applicable in 174 runs, and
 123 all 174 retained the intended task result, protected content, and valid Mu-
 124 sicXML. The remaining 66 tasks were not widened because the instruction
 125 did not name a specific measure, the source lacked an adjacent measure, or
 126 the operation was global. This is deterministic product evidence, not model
 127 accuracy.

Evidence	Scale	Verified result	Claim boundary
Benchmark	120 tasks	120/120 valid	Schema, Gold, con- straints, round trip
Regression	408 Python; 120 UI	All passed	Tested software envi- ronment
Reviewer demo	6 tasks	6/6; five host files	Offline product path
Product replay	720 runs	720/720; 660 exports	Local rules, not LLM accuracy
Host scope	240 runs	240/240; 174 widened	Target localization
Human review	120 + 30	Complete; zero stale	Same reviewer; no aes- thetic claim

Table 3: Triangulated release evidence and the boundary of each claim.

128 4. Impact

129 SeraEdit turns failure into inspectable research data. Rejections are classi-
 130 fied by stage, including malformed patches, invalid selectors, duration mis-
 131 matches, broken relations, protected-scope violations, incomplete execution,
 132 over-editing, conflicts, unsupported operations, and provider timeouts. Raw
 133 outputs and deterministic metrics enable studies of which safeguards affect
 134 validity, task completion, preservation, minimality, repair, and refusal.

135 Researchers can add public-domain scores and task constraints without re-
136 building a front end, provider wrapper, transaction engine, or reporting
137 pipeline. The offline path supports continuous integration, teaching, and
138 reproducibility; model-backed runs remain clearly labeled. A desktop and
139 MusicXML bridge expose the same contracts to composers and notation
140 users.

141 The source-bound scope/transaction pattern may also transfer to language-
142 guided edits of other structured scientific documents. Within music it sup-
143 plies a foundation for human studies without treating generated output as
144 ground truth. Current evidence supports reliability engineering and human-
145 checked benchmark semantics, not aesthetic improvement. Fixtures are small
146 and synthetic, and the repeat check used the same reviewer rather than an
147 independent panel. Complex tuplets, grace relations, cross-voice notation,
148 and universal host compatibility are not established. MuseScore has an ex-
149 ercised bridge; Sibelius is not claimed as verified. Provider behavior changes
150 with models and networks, while structural orchestration and unrestricted
151 composition remain outside the host-safe contract.

152 **5. Conclusions**

153 SeraEdit packages language-guided MusicXML editing as a source-bound,
154 scoped, validated, and reversible transaction. Models may propose, but
155 canonical score state, deterministic validators, protected-scope comparison,
156 round trips, and user review decide whether a revision exists. The open-
157 source release candidate includes a desktop demonstration, host bridge, bench-
158 mark, three conditions, automated metrics, tests, and reproducibility tooling.
159 Future work prioritizes formal model comparisons, cross-host tests, richer
160 notation relations, and independent blinded musician evaluation rather than
161 weakening the safeguards.

162 **Author contributions**

163 [AUTHOR NAME]: Conceptualization, Methodology, Software, Validation,
164 Investigation, Data curation, Writing – original draft, Writing – review &
165 editing, Visualization. Replace with the actual contributions of every author.

166 **Funding**

167 [FUNDING STATEMENT OR NO-SPECIFIC-FUNDING STATEMENT]

168 **Declaration of competing interest**

169 [INSERT THE AUTHORS' VERIFIED DECLARATION.]

170 **Data and code availability**

171 Source, CC0 benchmark fixtures, and the non-formal verification run will be
172 available from the tagged public repository and permanent archive in Table 1.

173 No API key or user score is distributed.

174 **References**

- 175 [1] M. Good (Ed.), MusicXML 4.0, W3C Music Notation Community
176 Group Final Report, 2021. [https://www.w3.org/2021/06/music](https://www.w3.org/2021/06/musicxml40/)
177 [xml40/](https://www.w3.org/2021/06/musicxml40/).
- 178 [2] M. S. Cuthbert, C. Ariza, music21: A toolkit for computer-aided mu-
179 sicology and symbolic music data, in: Proceedings of ISMIR, 2010, pp.
180 637–642.
- 181 [3] C. Raffel, D. P. W. Ellis, Intuitive analysis, creation and manipulation
182 of MIDI data with pretty_midi, in: ISMIR Late-Breaking and Demo
183 Papers, 2014.
- 184 [4] R. Yuan et al., ChatMusician: Understanding and generating music
185 intrinsically with LLM, in: Findings of ACL, 2024.
- 186 [5] Z. Zhou et al., Can LLMs “reason” in music? An evaluation of LLMs’
187 capability of music understanding and generation, in: Proceedings of
188 ISMIR, 2024.
- 189 [6] J. Guo et al., CodeEditorBench: Evaluating code editing capability of
190 large language models, arXiv:2404.03543 (2024).
- 191 [7] A. M. Smith, D. S. Katz, K. E. Niemeyer, FORCE11 Software Citation
192 Working Group, Software citation principles, PeerJ Computer Science 2
193 (2016) e86. <https://doi.org/10.7717/peerj-cs.86>.